

Alcance

Duración total estimada: 360 minutos. Los seis ejercicios básicos toman aproximadamente 12 minutos cada uno; los intermedios, 18 minutos; y los avanzados, 30 minutos. Use `new[]` y `delete[]` para el recurso principal; no use contenedores de la biblioteca estándar. Todos los identificadores de C++ se presentan en inglés. Los tiempos suponen que el estudiante conoce `new[]`, `delete[]` y `std::move`; no son tiempos de primera exposición a esos temas.

Los bloques siguientes usan `assert` (`#include <cassert>`). En cada ejercicio se deben implementar todos los métodos declarados en la clase.

Nivel básico

Ejercicio 1 (Básico · 12 min): `IntStack` y copia profunda

Contexto. Un evaluador de expresiones guarda los operandos pendientes en una pila. Antes de probar una alternativa, debe conservar una instantánea de la pila actual: una ruta puede retirar o agregar operandos sin cambiar el estado que necesitará la otra. Por ello, la pila original y su copia deben administrar arreglos independientes.

La clase `IntStack` modela esa pila: `data_` almacena los operandos, `size_` indica cuántos están activos y `capacity_` el espacio reservado.

Implemente todos los métodos de `IntStack`. Mantenga la invariante $0 \leq \text{size_} \leq \text{capacity_}$; el constructor de copia debe reservar su propio arreglo y copiar solo los elementos activos, no toda la capacidad disponible.

Idea clave. `size_` cuenta los valores almacenados y `capacity_` indica los espacios reservados. Al copiar una pila no basta copiar el puntero: cada objeto debe tener un arreglo propio para poder modificarse o destruirse sin afectar al otro.

```
class IntStack {
    int* data_;
    std::size_t size_;
    std::size_t capacity_;
public:
    explicit IntStack(std::size_t capacity = 8);
    IntStack(const IntStack& other);
    ~IntStack();
    void push(int value);
    void pop();
    int top() const;
    bool empty() const;
};
```

Caso de prueba:

```
IntStack source(4);
source.push(10);
source.push(20);
IntStack copy(source);
```

```

copy.pop();
copy.push(99);
assert(source.top() == 20);
assert(copy.top() == 99);

```

Caso adicional:

```

IntStack empty(2);
IntStack copy(empty);
assert(copy.empty());
copy.push(7);
assert(copy.top() == 7);
assert(empty.empty());

```

Ejercicio 2 (Básico · 12 min): GradeBook y asignación por copia

Contexto. Cada sección tiene su propio libro de calificaciones (GradeBook), almacenado en un arreglo dinámico. Al inicio de un nuevo periodo, una sección puede actualizar su libro con las notas registradas en otra sección. Esa actualización debe reemplazar las notas anteriores del destino sin crear un vínculo entre ambos libros: después, cada sección puede modificar sus notas de forma independiente.

La clase GradeBook representa el libro de una sección: `data_` contiene sus notas y `size_` la cantidad de notas almacenadas. La asignación `destination = source` representa la actualización descrita.

Implemente todos los métodos de GradeBook. En particular, el operador de asignación por copia debe soportar `a = a`, reservar primero la copia nueva y liberar el arreglo anterior solo después de tener una copia válida.

Idea clave. La asignación por copia reemplaza el estado de un objeto que ya existe. Por ello debe protegerse de `a = a`: el objeto origen y el destino son el mismo, y liberar el arreglo antes de copiarlo destruiría los datos fuente.

```

class GradeBook {
    double* data_;
    std::size_t size_;
public:
    explicit GradeBook(std::size_t size);
    GradeBook(const GradeBook& other);
    GradeBook& operator=(const GradeBook& other);
    ~GradeBook();
    double& at(std::size_t index);
    std::size_t size() const;
};

```

Caso de prueba:

```

GradeBook source(2);
source.at(0) = 15.0;
GradeBook copy(1);
copy = source;
copy.at(0) = 20.0;
copy = copy;
assert(source.at(0) == 15.0);
assert(copy.at(0) == 20.0);

```

Caso adicional:

```

GradeBook empty(0);
GradeBook target(2);
target = empty;
assert(target.size() == 0);

```

```
target = target;
assert(target.size() == 0);
```

Ejercicio 3 (Básico · 12 min): IntBuffer con crecimiento

Contexto. Un sensor envía lecturas enteras durante una jornada, pero no se conoce de antemano cuántas llegarán. El sistema las guarda en orden cronológico en un búfer que empieza con poco espacio. Cuando se llena, debe reservar más memoria sin perder ni reordenar las lecturas ya registradas.

La clase IntBuffer representa el búfer del sensor: `data_` guarda las lecturas, `size_` cuenta las recibidas y `capacity_` controla cuánto espacio hay reservado antes de tener que crecer.

Implemente todos los métodos de IntBuffer. Cuando se llene el arreglo, `pushBack` debe duplicar su capacidad y conservar los enteros en el mismo orden. Después de la realocación, libere exactamente una vez el arreglo anterior y actualice `capacity_` antes de aceptar el nuevo dato.

Idea clave. `size_` es la cantidad de lecturas válidas, mientras que `capacity_` es el tamaño del arreglo reservado. `resize` copia solo las posiciones desde 0 hasta `size_ - 1`; las demás no contienen lecturas.

```
class IntBuffer {
    int* data_;
    std::size_t size_;
    std::size_t capacity_;
    void resize(std::size_t newCapacity);
public:
    explicit IntBuffer(std::size_t capacity = 2);
    void pushBack(int value);
    int at(std::size_t index) const;
    std::size_t size() const;
    ~IntBuffer();
};
```

Caso de prueba:

```
IntBuffer buffer(1);
for (int value = 0; value < 5; ++value) buffer.pushBack(value * 10);
assert(buffer.size() == 5);
assert(buffer.at(0) == 0);
assert(buffer.at(4) == 40);
```

Caso adicional:

```
IntBuffer one(1);
one.pushBack(42);
assert(one.size() == 1);
assert(one.at(0) == 42);
```

Ejercicio 4 (Básico · 12 min): CircularQueue y orden lógico

Contexto. Una impresora recibe trabajos y los atiende en el mismo orden en que llegaron (FIFO). Para no desplazar todos los trabajos al terminar uno, la cola reutiliza las posiciones liberadas al inicio del arreglo. Después de varias inserciones y retiros, el primer trabajo pendiente puede estar en cualquier posición del arreglo, no necesariamente en `data_[0]`.

La clase CircularQueue representa la cola de impresión. `data_` contiene los trabajos, `front_` señala el primero pendiente, `size_` cuenta los trabajos y `capacity_` determina dónde se produce el retorno circular.

Implemente todos los métodos de CircularQueue. El constructor de copia debe preservar el orden de salida aunque `front_` no sea cero. La copia puede normalizar su `front_` a cero, pero el primer dequeue debe producir el mismo valor que en la cola fuente.

Idea clave. `front_` guarda la posición física del primer elemento lógico de la cola. Después de retirar un elemento no se desplaza todo el arreglo: se avanza `front_` con módulo de `capacity_`. Así, cuando el arreglo se envuelve, los elementos se recorren desde `data_[front_]`, no necesariamente desde `data_[0]`.

```
class CircularQueue {
    int* data_;
    std::size_t front_;
    std::size_t size_;
    std::size_t capacity_;
public:
    explicit CircularQueue(std::size_t capacity);
    CircularQueue(const CircularQueue& other);
    void enqueue(int value);
    int dequeue();
    bool empty() const;
};
```

Caso de prueba:

```
CircularQueue source(3);
source.enqueue(1); source.enqueue(2); source.enqueue(3);
assert(source.dequeue() == 1);
source.enqueue(4);
CircularQueue copy(source);
assert(copy.dequeue() == 2);
assert(copy.dequeue() == 3);
assert(copy.dequeue() == 4);
```

Caso adicional:

```
CircularQueue empty(2);
CircularQueue copy(empty);
assert(copy.empty());
copy.enqueue(7);
assert(copy.dequeue() == 7);
assert(empty.empty());
```

Ejercicio 5 (Básico · 12 min): Name y copia de `char*`

Contexto. Una ficha de estudiante guarda un nombre de longitud variable en memoria dinámica. Al crear una ficha de respaldo, ambas deben comenzar con el mismo texto, pero luego deben poder corregirse por separado. Si las dos fichas compartieran el mismo arreglo de caracteres, cambiar o destruir una afectaría a la otra.

La clase `Name` modela la ficha por medio de `text_`, un arreglo dinámico que contiene el nombre. Una instancia de `Name` es una ficha; su constructor de copia crea la ficha de respaldo.

Implemente todos los métodos de `Name`, incluido `duplicate`. Para copiar un nombre almacenado en un único `char*`, reserve espacio para todos los caracteres más el terminador `\0`; nunca copie el puntero `text_` directamente.

Idea clave. Una cadena de C termina en `\0`. Dos objetos que guardan la misma dirección en `text_` comparten el texto y ambos intentarían liberar el mismo arreglo; `duplicate` evita ese problema al crear una nueva copia de la cadena.

```
class Name {
    char* text_;
    static char* duplicate(const char* text);
public:
    explicit Name(const char* text);
    Name(const Name& other);
    ~Name();
    void setFirst(char value);
};
```

```
const char* cStr() const;
};
```

Caso de prueba:

```
Name source("Ana");
Name copy(source);
copy.setFirst('L');
assert(std::strcmp(source.cStr(), "Ana") == 0);
assert(std::strcmp(copy.cStr(), "Lna") == 0);
```

Caso adicional:

```
Name empty("");
Name copy(empty);
assert(std::strcmp(empty.cStr(), "") == 0);
assert(std::strcmp(copy.cStr(), "") == 0);
```

Ejercicio 6 (Básico · 12 min): AttendanceRecord y asignación encadenada

Contexto. Un sistema mantiene registros de asistencia para varios grupos. Al usar un registro como plantilla para otro, el destino debe reemplazar todas sus marcas anteriores por las del origen. Luego cada grupo puede actualizar su asistencia sin modificar la del otro; además, la operación debe seguir siendo válida cuando se encadenan varias asignaciones.

La clase AttendanceRecord modela un registro de grupo: present_ guarda una marca por estudiante y count_ indica cuántas marcas contiene. El operador = representa copiar una plantilla de asistencia a otro grupo.

Implemente todos los métodos de AttendanceRecord. El operador de asignación debe hacer que third = second = first funcione, devolver AttendanceRecord&, detectar autoasignación y copiar las count_ marcas sin compartir el arreglo present_.

Idea clave. La expresión third = second = first se evalúa de derecha a izquierda. Por eso second = first debe devolver una referencia a second, que luego puede convertirse en el origen de la segunda asignación.

```
class AttendanceRecord {
    bool* present_;
    std::size_t count_;
public:
    explicit AttendanceRecord(std::size_t count);
    AttendanceRecord(const AttendanceRecord& other);
    AttendanceRecord& operator=(const AttendanceRecord& other);
    ~AttendanceRecord();
    void mark(std::size_t student, bool value);
    bool isPresent(std::size_t student) const;
};
```

Caso de prueba:

```
AttendanceRecord first(2), second(2), third(2);
first.mark(0, true);
third = second = first;
second = second;
assert(second.isPresent(0));
assert(third.isPresent(0));
```

Caso adicional:

```
second.mark(0, false);
assert(!second.isPresent(0));
```

```
assert(first.isPresent(0));  
assert(third.isPresent(0));
```

Nivel intermedio

Ejercicio 7 (Intermedio · 18 min): Text y movimiento

Contexto. Un editor construye un mensaje en un objeto temporal y, al enviarlo, lo transfiere a una cola de salida. El texto ya tiene un arreglo dinámico propio; duplicarlo para transferirlo sería innecesario. La cola debe recibir ese arreglo y el objeto temporal debe quedar vacío, pero aún válido para poder destruirse o reutilizarse con seguridad.

La clase `Text` modela cada mensaje: `chars_` es el arreglo que contiene el texto y `length_` su longitud. Construir o asignar desde un `Text&&` representa la transferencia del mensaje a la cola.

Implemente todos los métodos de `Text`, incluida la regla de cinco. Las dos operaciones de movimiento deben transferir el puntero y dejar el origen vacío y válido. La asignación por movimiento debe liberar primero el recurso de `*this`, salvo en auto-movimiento.

Idea clave. Copiar crea un nuevo arreglo de caracteres; mover transfiere la propiedad del arreglo existente. Después de mover, el origen debe quedar en un estado válido y destruible, por ejemplo con `chars_ == nullptr` y longitud cero.

```
class Text {
    char* chars_;
    std::size_t length_;
public:
    Text();
    explicit Text(const char* source);
    ~Text();
    Text(const Text& other);
    Text& operator=(const Text& other);
    Text(Text&& other) noexcept;
    Text& operator=(Text&& other) noexcept;
    const char* cStr() const;
    bool empty() const;
};
```

Caso de prueba:

```
Text source("UTEC");
Text moved(std::move(source));
assert(std::strcmp(moved.cStr(), "UTEC") == 0);
assert(source.empty());
Text target("old");
target = std::move(moved);
assert(std::strcmp(target.cStr(), "UTEC") == 0);
assert(moved.empty());
```

Caso adicional:

```
Text empty;
Text copy(std::move(empty));
assert(empty.empty());
assert(copy.empty());
```

Ejercicio 8 (Intermedio · 18 min): IntMatrix y swap

Contexto. Un módulo de cálculo produce matrices temporales, por ejemplo el resultado de una operación, y luego reemplaza la matriz de trabajo por ese resultado. Copiar cada celda al reemplazarla puede ser costoso. La matriz destino debe adoptar directamente el bloque dinámico de la temporal, mientras que la temporal queda sin datos y con dimensiones cero.

La clase `IntMatrix` representa cada matriz: `data_` guarda sus celdas y `rows_` junto con `columns_` describe su forma. Las operaciones de movimiento representan el reemplazo de la matriz de trabajo por la matriz temporal.

Implemente todos los métodos de `IntMatrix`. `swap`, el constructor de movimiento y la asignación por movimiento deben intercambiar o transferir únicamente el puntero y las dimensiones; no copie los valores de la matriz. Tras mover, el origen debe tener dimensiones cero.

Idea clave. Una matriz se identifica completamente por su puntero, filas y columnas. Intercambiar esos tres miembros cambia la propiedad de todo el bloque de datos en tiempo constante; recorrer y copiar las celdas no es un movimiento.

```
class IntMatrix {
    int* data_;
    std::size_t rows_;
    std::size_t columns_;
    void swap(IntMatrix& other) noexcept;
public:
    IntMatrix(std::size_t rows, std::size_t columns);
    IntMatrix(IntMatrix&& other) noexcept;
    IntMatrix& operator=(IntMatrix&& other) noexcept;
    std::size_t rows() const;
    std::size_t columns() const;
};
```

Caso de prueba:

```
IntMatrix source(2, 3);
IntMatrix target(1, 1);
target = std::move(source);
assert(target.rows() == 2 && target.columns() == 3);
assert(source.rows() == 0 && source.columns() == 0);
```

Caso adicional:

```
IntMatrix empty(0, 0);
IntMatrix moved(std::move(empty));
assert(moved.rows() == 0 && moved.columns() == 0);
assert(empty.rows() == 0 && empty.columns() == 0);
```

Ejercicio 9 (Intermedio · 18 min): Polynomial y retorno por valor

Contexto. Un módulo simbólico representa una trayectoria mediante un polinomio y necesita su derivada para calcular la velocidad. La trayectoria original sigue siendo necesaria para otros cálculos, por lo que derivarla no puede modificar sus coeficientes. En su lugar, la operación debe construir y devolver un segundo polinomio con los coeficientes de la derivada.

La clase `Polynomial` modela la ecuación: `coefficients_` guarda sus coeficientes y `degree_` su mayor exponente. El método `derivative` representa el cálculo de la velocidad y devuelve un nuevo objeto `Polynomial`.

Implemente todos los métodos de `Polynomial`. `derivative` debe crear y devolver un nuevo polinomio sin modificar el objeto original. Si $p(x) = a_0 + a_1x + \dots + a_nx^n$, el coeficiente $i - 1$ del resultado debe ser $i * a_i$. Trate el polinomio constante como un caso especial válido.

Idea clave. La posición del arreglo es el exponente: `coefficients_[0]` es el término constante. La derivada desplaza cada coeficiente una posición hacia la izquierda; la derivada de un polinomio constante se representa como el polinomio cero.

```
class Polynomial {
    double* coefficients_;
    std::size_t degree_;
public:
    explicit Polynomial(std::size_t degree);
    Polynomial derivative() const;
    double evaluate(double x) const;
```



```
double& coefficient(std::size_t exponent);
};
```

Caso de prueba:

```
Polynomial source(2); // 3x^2 + 2x + 1
source.coefficient(0) = 1;
source.coefficient(1) = 2;
source.coefficient(2) = 3;
Polynomial derived = source.derivative();
assert(source.evaluate(2) == 17);
assert(derived.evaluate(2) == 14); // 6x + 2
```

Caso adicional:

```
Polynomial constant(0);
constant.coefficient(0) = 8;
Polynomial zero = constant.derivative();
assert(zero.evaluate(0) == 0);
assert(zero.evaluate(5) == 0);
```

Ejercicio 10 (Intermedio · 18 min): GrayImage y objeto temporal

Contexto. Una aplicación conserva la imagen original y genera vistas filtradas para mostrarla. Al invertir los tonos de gris, cada píxel de la nueva imagen se calcula a partir del original, que debe permanecer intacto. La imagen resultante puede ser temporal y luego transferirse a otra variable sin copiar nuevamente todo su arreglo de píxeles.

La clase GrayImage representa una imagen: `pixels_` contiene los tonos y `width_` y `height_` definen sus dimensiones. `inverted` modela el filtro y las operaciones de movimiento modelan la transferencia de la imagen temporal.

Implemente todos los métodos de GrayImage. Para cada píxel `p`, `inverted` debe almacenar $255 - p$ en una imagen nueva; el método no puede alterar la imagen original. Las operaciones de movimiento deben transferir el arreglo y dejar el origen vacío y válido.

Idea clave. La sobrecarga no constante de `at` permite escribir un píxel y la sobrecarga constante permite leerlo. Para ubicar `(row, column)` en un arreglo lineal de una imagen de ancho `width_`, use el desplazamiento `row * width_ + column`.

```
class GrayImage {
    unsigned char* pixels_;
    std::size_t width_;
    std::size_t height_;
public:
    GrayImage(std::size_t width, std::size_t height);
    GrayImage(GrayImage&& other) noexcept;
    GrayImage& operator=(GrayImage&& other) noexcept;
    GrayImage inverted() const;
    unsigned char& at(std::size_t row, std::size_t column);
    unsigned char at(std::size_t row, std::size_t column) const;
    bool empty() const;
};
```

Caso de prueba:

```
GrayImage source(1, 1);
source.at(0, 0) = 10;
GrayImage result = source.inverted();
assert(source.at(0, 0) == 10);
assert(result.at(0, 0) == 245);
```

```

GrayImage target(1, 1);
target = std::move(result);
assert(target.at(0, 0) == 245 && result.empty());

```

Caso adicional:

```

GrayImage white(1, 1);
white.at(0, 0) = 255;
GrayImage black = white.inverted();
assert(white.at(0, 0) == 255);
assert(black.at(0, 0) == 0);

```

Ejercicio 11 (Intermedio · 18 min): ScoreList y movimiento explícito

Contexto. Un generador crea una lista local de puntajes de muestra y la entrega al programa principal como resultado de una función. Al retornar, esa lista deja de ser necesaria dentro del generador, pero el receptor debe conservar sus datos. Este escenario permite observar si el compilador construye el resultado directamente en el receptor o si transfiere el recurso mediante movimiento.

La clase `ScoreList` representa la lista construida por el generador: `scores_` almacena los puntajes y `size_` y `capacity_` controlan su contenido y espacio. La función `makeSample` crea y devuelve una instancia de esta clase.

Implemente todos los métodos de `ScoreList` y defina `makeSample`, que devuelve una lista por valor. Inserte tres valores dentro de esa función e instrumente las operaciones de movimiento con un mensaje para identificar si hay elisión de copia o movimiento al inicializar el receptor.

Idea clave. Al devolver un objeto local por valor, el compilador puede construir el resultado directamente en el receptor (elisión de copia) o usar una operación de movimiento. La instrumentación permite observar cuál ocurre en cada caso.

```

class ScoreList {
    int* scores_;
    std::size_t size_;
    std::size_t capacity_;
public:
    ScoreList();
    void add(int score);
    std::size_t size() const;
};

```

Caso de prueba:

```

ScoreList makeSample() {
    ScoreList result;
    result.add(10);
    result.add(20);
    return result;
}
ScoreList scores = makeSample();
assert(scores.size() == 2);

```

Caso adicional:

```

void consume(ScoreList value) {
    assert(value.size() == 2);
}
consume(makeSample());

```

Ejercicio 12 (Intermedio · 18 min): CourseRoster y copia segura

Contexto. Un curso mantiene en un arreglo dinámico los identificadores de sus estudiantes, que puede estar vacío o contener una lista activa. Al actualizar un curso con la lista de otro, la reserva de memoria para la nueva lista puede fallar. En ese caso, el curso destino no debe perder la lista que ya tenía; swap permite realizar la actualización solo cuando la copia nueva existe.

La clase CourseRoster modela esa lista: `ids_` almacena los identificadores y `size_` indica su cantidad. La asignación por copia representa actualizar el roster de un curso con el de otro.

Implemente todos los métodos de CourseRoster, incluido `swap` y la asignación por copia con una copia temporal. El temporal debe adquirir la copia de `other`; luego `swap` deja a `*this` actualizado y el temporal destruye el arreglo antiguo al salir del operador.

Idea clave. La copia temporal se construye antes de modificar el destino. Si la reserva falla, el destino aún conserva su arreglo anterior; si tiene éxito, `swap` intercambia ambos arreglos sin lanzar excepciones.

```
class CourseRoster {
    int* ids_;
    std::size_t size_;
    void swap(CourseRoster& other) noexcept;
public:
    explicit CourseRoster(std::size_t size = 0);
    CourseRoster& operator=(const CourseRoster& other);
    std::size_t size() const;
};
```

Caso de prueba:

```
CourseRoster empty;
CourseRoster copy(3);
copy = empty;
assert(copy.size() == 0);
copy = copy;
assert(copy.size() == 0);
```

Caso adicional:

```
CourseRoster source(4);
CourseRoster target(2);
target = source;
assert(target.size() == 4);
assert(source.size() == 4);
```

Nivel avanzado

Ejercicio 13 (Avanzado · 30 min): History para lvalue y rvalue

Contexto. Un historial registra mensajes de eventos. A veces recibe un mensaje que el programa seguirá usando después de guardarlo; otras veces recibe un mensaje temporal creado solo para el historial. En el primer caso, el historial debe conservar una copia propia; en el segundo, puede apropiarse de los recursos del mensaje para evitar una copia innecesaria.

La clase History modela el registro: `entries_` es el arreglo de mensajes, `size_` cuenta los eventos guardados y `capacity_` su espacio reservado. Las dos sobrecargas de `add` representan las dos formas de recibir un `Text`.

Use `Text` como elemento y un arreglo dinámico como almacenamiento. Implemente todos los métodos de `History`, incluidas las dos sobrecargas de `add`. `add(const Text&)` debe conservar el mensaje del llamador; `add(Text&&)` debe usar `std::move` para construir la nueva entrada. Ambas deben incrementar `size_` exactamente una vez.

Idea clave. Un lvalue representa un mensaje que seguirá siendo usado por el llamador, por lo que se copia. Un rvalue representa un objeto temporal o cedido, por lo que se mueve. Las sobrecargas expresan esta diferencia de propiedad.

```
class History {
    Text* entries_;
    std::size_t size_;
    std::size_t capacity_;
public:
    History();
    void add(const Text& entry);
    void add(Text&& entry);
    std::size_t size() const;
    const Text& at(std::size_t index) const;
};
```

Caso de prueba:

```
History history;
Text message("start");
history.add(message);
history.add(Text("stop"));
history.add(std::move(message));
assert(history.size() == 3);
assert(std::strcmp(history.at(0).cStr(), "start") == 0);
assert(std::strcmp(history.at(1).cStr(), "stop") == 0);
```

Caso adicional:

```
Text reusable("keep");
history.add(reusable);
assert(std::strcmp(reusable.cStr(), "keep") == 0);
assert(std::strcmp(history.at(3).cStr(), "keep") == 0);
```

Ejercicio 14 (Avanzado · 30 min): TaskList y separación de resultados

Contexto. Al cierre del día, una aplicación separa las tareas completadas de las que siguen pendientes. Las completadas se entregan en una nueva lista para archivarlas y las pendientes permanecen en la lista original para el día siguiente. En ambos resultados debe mantenerse el orden en que se registraron las tareas.

La clase `TaskList` modela cada una de esas listas: `tasks_` guarda las tareas, `size_` indica cuántas hay y `capacity_` el espacio disponible. El método `extractCompleted` representa la separación y devuelve la lista para archivar.

Implemente todos los métodos de `TaskList`. `extractCompleted` debe devolver las tareas terminadas y eliminarlas del original, preservando el orden. Recorra las tareas una sola vez para contar o clasificar; la lista devuelta y la lista original deben poseer arreglos distintos.

Idea clave. Separar las tareas no consiste solo en devolver una lista nueva: también debe compactarse la lista original con las pendientes. Los dos resultados deben tener almacenamiento independiente para evitar que uno invalide al otro.

```
struct Task {
    Text title;
    bool completed;
};
class TaskList {
    Task* tasks_;
    std::size_t size_;
    std::size_t capacity_;
public:
    TaskList();
    TaskList extractCompleted();
    void add(Text&& title);
    void complete(std::size_t index);
    std::size_t size() const;
    const Task& at(std::size_t index) const;
};
```

Caso de prueba:

```
TaskList pending;
pending.add(Text("code"));
pending.add(Text("test"));
pending.complete(1);
TaskList done = pending.extractCompleted();
assert(pending.size() == 1 && done.size() == 1);
assert(std::strcmp(done.at(0).title.cStr(), "test") == 0);
```

Caso adicional:

```
TaskList pendingOnly;
pendingOnly.add(Text("read"));
TaskList none = pendingOnly.extractCompleted();
assert(pendingOnly.size() == 1);
assert(none.size() == 0);
```

Ejercicio 15 (Avanzado · 30 min): TextList y realocación con movimiento

Contexto. Un chat conserva sus mensajes en una lista que puede quedarse sin capacidad cuando llegan más mensajes de los previstos. Al ampliar el arreglo de Text, los mensajes ya almacenados deben seguir en el mismo orden. Como cada Text posee su propio arreglo de caracteres, trasladarlos mediante movimiento evita duplicar innecesariamente el contenido de todos los mensajes.

La clase TextList representa la conversación almacenada: data_ es el arreglo de mensajes, size_ su cantidad y capacity_ el espacio reservado. resize representa la ampliación de la conversación y add la llegada de un mensaje nuevo.

Implemente todos los métodos de TextList, incluido el crecimiento y ambas sobrecargas de add. Al crecer, traslade los elementos con std::move. El nuevo arreglo debe tener la capacidad pedida y, tras mover cada elemento, debe liberarse el arreglo antiguo. La sobrecarga rvalue no debe crear una copia intermedia del argumento.

Idea clave. Aunque el arreglo de Text cambia de dirección al crecer, cada mensaje puede transferir sus caracteres al nuevo objeto mediante movimiento. Después de mover todos los elementos, el arreglo antiguo contiene objetos vacíos y puede destruirse con seguridad.

```
class TextList {
    Text* data_;
    std::size_t size_;
```

```

    std::size_t capacity_;
    void resize(std::size_t newCapacity);
public:
    explicit TextList(std::size_t capacity = 2);
    void add(const Text& value);
    void add(Text&& value);
    std::size_t size() const;
    const Text& at(std::size_t index) const;
};

```

Caso de prueba:

```

TextList list(1);
Text first("one");
list.add(first);
list.add(Text("two"));
list.add(Text("three"));
assert(list.size() == 3);
assert(std::strcmp(list.at(2).cStr(), "three") == 0);

```

Caso adicional:

```

Text external("outside");
TextList check(1);
check.add(external);
check.add(Text("next"));
assert(std::strcmp(external.cStr(), "outside") == 0);
assert(std::strcmp(check.at(0).cStr(), "outside") == 0);

```

Ejercicio 16 (Avanzado · 30 min): AdjacencyMatrix y transposición

Contexto. Un grafo dirigido describe llamadas entre servicios: una arista $u \rightarrow v$ significa que el servicio u llama al servicio v . La matriz original facilita consultar a quién llama un servicio. Para responder quiénes llaman a un servicio, se construye un segundo grafo con todas las direcciones invertidas, donde cada arista $u \rightarrow v$ pasa a ser $v \rightarrow u$.

La clase `AdjacencyMatrix` modela el grafo: `edges_` guarda la matriz de conexiones y `vertices_` su número de servicios. El método `transposed` produce el segundo grafo que permite responder consultas en la dirección inversa.

Use un grafo dirigido pequeño con un arreglo dinámico de `bool` e implemente todos los métodos de `AdjacencyMatrix`. `transposed` debe devolver las aristas invertidas: para cada posición $(from, to)$ verdadera, la posición $(to, from)$ del nuevo grafo también debe ser verdadera. No modifique `edges_` del objeto fuente.

Idea clave. La matriz codifica una arista $from \rightarrow to$ en una celda. Transponer la matriz intercambia fila y columna: una arista de u a v se registra en el resultado como una arista de v a u .

```

class AdjacencyMatrix {
    bool* edges_;
    std::size_t vertices_;
public:
    explicit AdjacencyMatrix(std::size_t vertices);
    void connect(std::size_t from, std::size_t to);
    AdjacencyMatrix transposed() const;
    bool connected(std::size_t from, std::size_t to) const;
};

```

Caso de prueba:

```

AdjacencyMatrix graph(3);
graph.connect(0, 2);

```

```
AdjacencyMatrix transposed = graph.transposed();
assert(graph.connected(0, 2));
assert(!graph.connected(2, 0));
assert(transposed.connected(2, 0));
```

Caso adicional:

```
AdjacencyMatrix empty(2);
AdjacencyMatrix reversed = empty.transposed();
assert(!reversed.connected(0, 1));
assert(!reversed.connected(1, 0));
```

Ejercicio 17 (Avanzado · 30 min): FileBuffer como tipo solo movable

Contexto. Un búfer representa el contenido temporal de un archivo grande cargado en memoria. Ese bloque debe tener un único propietario responsable de liberarlo. Si dos objetos compartieran el mismo puntero por una copia accidental, ambos creerían que deben liberarlo; transferir el búfer entre etapas del programa requiere, por tanto, movimiento y no copia.

La clase FileBuffer representa ese bloque temporal: `bytes_` apunta a los datos y `size_` registra cuántos bytes posee. Sus operaciones de movimiento representan la transferencia del único propietario entre dos etapas.

Modele e implemente todos los métodos de un búfer de bytes de propietario único. Elimine las operaciones de copia e implemente las dos de movimiento. No realice lectura de archivos. El movimiento debe transferir `bytes_`, copiar `size_` al destino y reiniciar el origen con `nullptr` y tamaño cero. La asignación por movimiento debe liberar primero el búfer que ya posee el destino.

Idea clave. Un tipo de propietario único tiene exactamente un objeto responsable de liberar el recurso. Mover cambia quién es ese propietario; copiar se prohíbe para que dos objetos no intenten liberar el mismo búfer.

```
class FileBuffer {
    char* bytes_;
    std::size_t size_;
public:
    explicit FileBuffer(std::size_t size);
    FileBuffer(const FileBuffer&) = delete;
    FileBuffer& operator=(const FileBuffer&) = delete;
    FileBuffer(FileBuffer&& other) noexcept;
    FileBuffer& operator=(FileBuffer&& other) noexcept;
    ~FileBuffer();
    std::size_t size() const;
};
```

Caso de prueba:

```
FileBuffer source(128);
FileBuffer target(std::move(source));
assert(target.size() == 128);
assert(source.size() == 0);
target = std::move(target); // auto-movimiento defensivo
assert(target.size() == 128);
```

Caso adicional:

```
FileBuffer replacement(64);
target = std::move(replacement);
assert(target.size() == 64);
assert(replacement.size() == 0);
```

Ejercicio 18 (Avanzado · 30 min): DocumentIndex y fusión por movimiento

Contexto. Dos documentos construyen por separado sus índices de palabras y luego se consolidan en un único índice. El segundo índice ya no se usará después de la consolidación, por lo que puede entregar sus entradas al primero. La fusión debe conservar primero las palabras del destino, añadir las del origen en orden y dejar el índice transferido vacío y válido.

La clase DocumentIndex representa cada índice: words_ almacena las palabras, size_ cuenta las entradas y capacity_ el espacio reservado. merge modela la consolidación al recibir el segundo DocumentIndex como rvalue.

Implemente todos los métodos de DocumentIndex, incluido merge, que toma un rvalue, crece solo si es necesario e incorpora sus entradas. El origen queda vacío y válido. Preserve primero las palabras ya existentes en *this; luego traslade en orden las entradas de other y actualice ambos tamaños. Debe funcionar si other está vacío.

Idea clave. merge recibe un rvalue porque el índice origen entrega sus palabras al destino. Tras mover cada entrada, ajuste other.size_ a cero para que su destructor no intente administrar elementos que ya fueron transferidos.

```
class DocumentIndex {
    Text* words_;
    std::size_t size_;
    std::size_t capacity_;
public:
    DocumentIndex();
    void add(Text&& word);
    void merge(DocumentIndex&& other);
    std::size_t size() const;
    const Text& at(std::size_t index) const;
};
```

Caso de prueba:

```
DocumentIndex first, second;
first.add(Text("cat"));
second.add(Text("dog"));
first.merge(std::move(second));
assert(first.size() == 2);
assert(second.size() == 0);
assert(std::strcmp(first.at(1).cStr(), "dog") == 0);
```

Caso adicional:

```
DocumentIndex empty;
std::size_t oldSize = first.size();
first.merge(std::move(empty));
assert(first.size() == oldSize);
assert(empty.size() == 0);
```

Lista de verificación

- Cada clase libera exactamente el recurso que posee.
- Las copias son profundas y la autoasignación es segura.
- Los movimientos dejan el origen válido y destruible.
- Las pruebas incluyen al menos un lvalue y un rvalue cuando el ejercicio lo requiere.